



git

D'où vient le nom ?

« *Git* » peut se traduire par « connard » en argot britannique.
Citation de Linus Torvald (le créateur de Git) : « Je ne suis qu'un sale égoцентриque, donc je nomme tous mes projets d'après ma propre personne. D'abord Linux, puis Git⁷. »



Définition

Git est un [logiciel de gestion de versions décentralisé](#). C'est un [logiciel libre](#) et gratuit, créé en 2005 par [Linus Torvalds](#).

Depuis les années 2010, il s'agit du [logiciel de gestion de versions](#) le plus populaire dans le développement logiciel et web. Il est utilisé par 12 millions de personnes.



Logiciels de gestion de versions

Un **logiciel de gestion de versions** (ou **VCS** en **anglais**, pour *version control system*) est un **logiciel** qui permet de stocker un ensemble de **fichiers** en conservant la chronologie de toutes les modifications qui ont été effectuées dessus.

A quoi ça sert ?

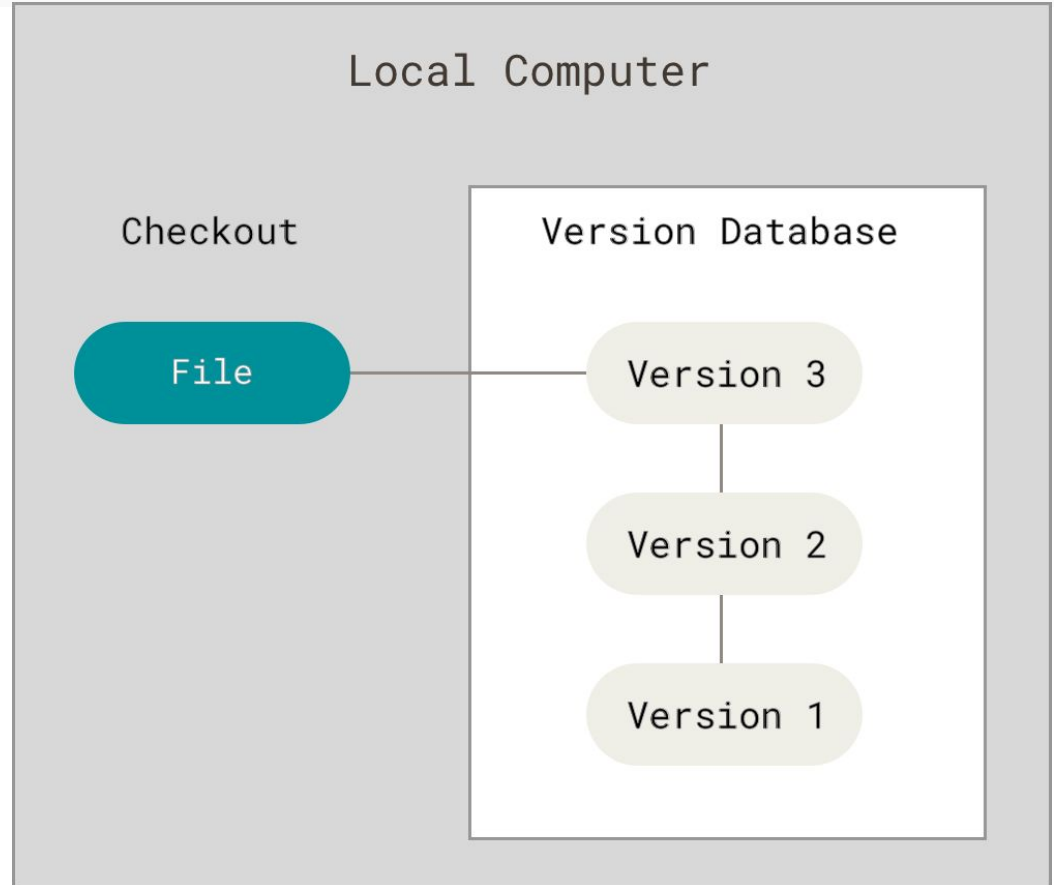
Lorsque plusieurs développeurs travaillent simultanément sur une même source, un VCS permet :

- de conserver les modifications de chacun et d'éviter qu'elles ne soient écrasées lors de leur partage
- que les développeurs puissent travailler sur les mêmes sections de code

Systèmes locaux anciens

Il y a longtemps, les développeurs ont développé des systèmes locaux qui utilisaient une base de données simple pour conserver les différentes versions d'un fichier en local.

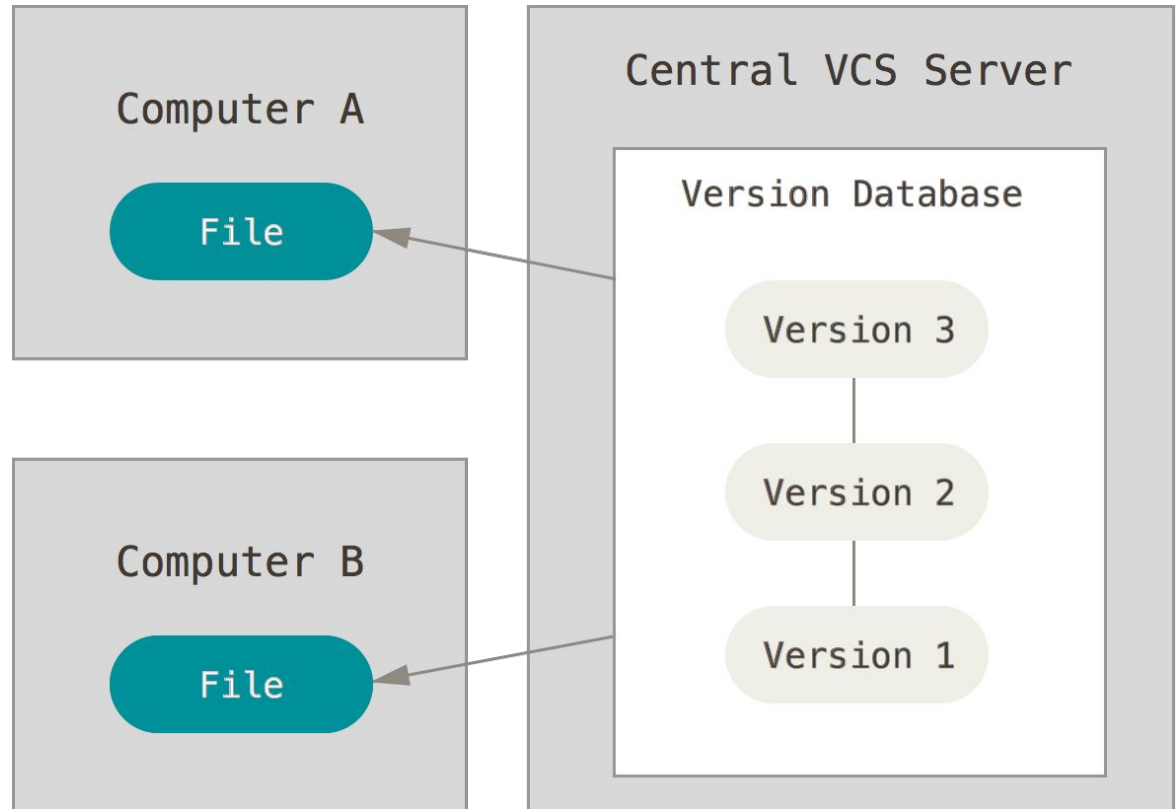
Le problème majeur qu'ils rencontraient avec ce système, était de pouvoir collaborer avec d'autres développeurs.



Systèmes de gestion de version centralisés

Pour traiter ce problème, les systèmes de gestion de version centralisés (CVCS en anglais pour *Centralized Version Control Systems*) ont été développés.

Ces systèmes (tels que CVS, Subversion, et Perforce) mettent en place un serveur central qui contient tous les fichiers sous gestion de version, et des clients qui peuvent extraire les fichiers de ce dépôt central.



Les systèmes de gestion de versions centralisés reposent sur un **serveur central**.

Cela crée un **point unique de panne** (*single point of failure*) :

- si le serveur est indisponible, les utilisateurs ne peuvent plus collaborer ;
- ils ne peuvent plus récupérer ou envoyer leurs modifications ;
- certaines opérations nécessitant le serveur deviennent impossibles.

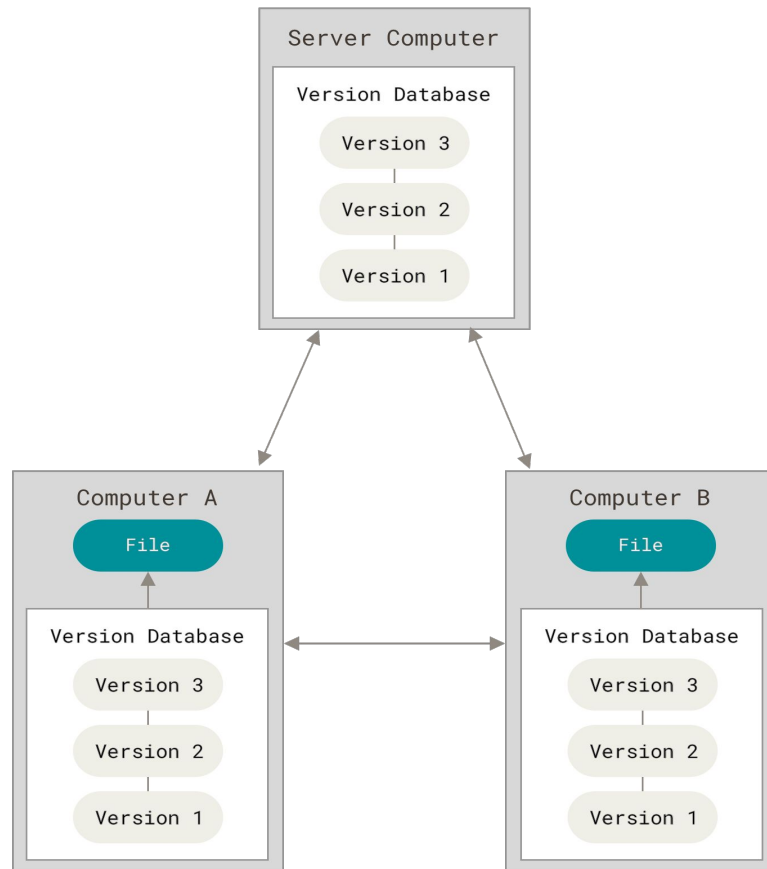
Git adopte une approche différente : chaque développeur possède une copie complète du dépôt et de son historique sur son ordinateur.

C'est pour cette raison que les **systèmes de gestion de versions distribués** (DVCS — *Distributed Version Control Systems*) ont été développés.

Dans un DVCS comme **Git**, chaque utilisateur possède une **copie complète du dépôt**, et pas seulement la dernière version des fichiers.

Cette copie contient **l'historique complet du projet**.

Ainsi, si le serveur central disparaît, le dépôt peut être restauré à partir de la copie d'un autre utilisateur.



Installer Git

Git peut être téléchargé depuis le site officiel :

<https://git-scm.com/download>

Installez la version correspondant à votre système d'exploitation.

Configuration utilisateur

Après l'installation de Git, configurez votre **nom et votre adresse e-mail**.

Ces informations seront associées à vos **commits** et permettront d'identifier leur auteur.

```
git config --global user.name "John Doe"
```

```
git config --global user.email "johndoe@example.com"
```

Utilisez votre propre nom et votre adresse e-mail.

Qu'est-ce qu'un dépôt Git ?

Un **dépôt Git** (*repository*) est un projet informatique géré par Git.

Il contient notamment :

- les **fichiers du projet** ;
- l'**historique des modifications** et des commits ;
- les informations nécessaires au fonctionnement de Git, stockées dans le dossier caché **.git**.

Vous pouvez créer un dépôt Git de deux manières principales :

1. **Initialiser un dépôt à partir d'un projet existant** avec `git init`.
2. **Cloner un dépôt existant** depuis un dépôt distant avec `git clone`.

Dans les deux cas, vous obtenez un **dépôt Git local** sur votre ordinateur.

Initialiser un dépôt Git dans un projet existant

Si vous avez déjà un projet sur votre ordinateur et que vous souhaitez commencer à le gérer avec Git :

1. Placez-vous dans le répertoire du projet :
`cd C:/Users/user/my_project`
2. Initialisez le dépôt Git :
`git init`

La commande `git init` crée un dossier caché `.git` dans le projet.

Le projet est maintenant un **dépôt Git local**. Vous pouvez commencer à suivre ses fichiers et à créer des commits.

Si un dépôt Git existe déjà sur un serveur distant, vous pouvez en obtenir une **copie locale** avec la commande :

```
git clone URL_DU_DEPOT
```

Par exemple :

```
git clone https://github.com/libgit2/libgit2.git
```

La commande `git clone` :

- télécharge les fichiers du projet ;
- récupère son **historique Git** ;
- crée un **dépôt Git local** sur votre ordinateur ;

Voir le dépôt distant avec Git remote

Après avoir cloné un dépôt, votre dépôt local est associé à un **dépôt distant**.

Pour afficher les dépôts distants associés à votre dépôt local, utilisez :

```
git remote -v
```

Vous devriez obtenir quelque chose comme :

```
origin https://github.com/... (fetch)
```

```
origin https://github.com/... (push)
```

origin est le **nom par défaut** donné au dépôt distant lors d'un *git clone*.

- **fetch** : récupérer des informations et des commits depuis le dépôt distant
- **push** : envoyer vos commits vers le dépôt distant

Git remote -v

```
tomezequiel@macbook-air-de-tom Atelier Git % git remote -v  
origin  git@github.com:tomMizEnseignement/atelierGitNexa15SeptTest.git (fetch)  
origin  git@github.com:tomMizEnseignement/atelierGitNexa15SeptTest.git (push)  
tomezequiel@macbook-air-de-tom Atelier Git %
```


L'outil principal pour connaître l'état des fichiers de votre dépôt est la commande `git status`.

Si vous exécutez cette commande juste après avoir cloné le dépôt, vous devriez obtenir un message similaire à celui-ci :

nothing to commit, working tree clean

Cela signifie que votre **répertoire de travail est propre** : aucun fichier n'a été modifié depuis le dernier commit.

```
[tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
tomezequiel@macbook-air-de-tom Atelier Git %
```

Créer un nouveau fichier

Si vous créez un nouveau fichier LISEZMOI et exécutez :

```
git status
```

Git l'affiche dans la section « **Untracked files** » (fichiers non suivis).

Cela signifie que **Git ne suit pas encore ce fichier** : il ne sera pas inclus dans un commit tant que vous ne l'aurez pas ajouté à la zone de staging.

```
tomezequiel@macbook-air-de-tom Atelier Git % echo 'mon projet' > LISEZMOI
tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  LISEZMOI

nothing added to commit but untracked files present (use "git add" to track)
tomezequiel@macbook-air-de-tom Atelier Git %
```

Ajouter un nouveau fichier à la zone de staging avec git add

Pour commencer à suivre un nouveau fichier et le préparer pour le prochain commit, utilisez `git add` :

```
git add LISEZMOI
```

Si vous exécutez à nouveau :

```
git status
```

le fichier apparaît maintenant dans la section « **Changes to be committed** » : il est **suivi par Git et placé dans la zone de staging**.

```
[tomezequiel@macbook-air-de-tom Atelier Git % git add LISEZMOI
[tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   LISEZMOI

tomezequiel@macbook-air-de-tom Atelier Git %
```

Rajouter un fichier à la zone de staging

Nouveau fichier LISEZMOI



git status



Untracked files



| git add LISEZMOI



Staging area

Ajouter un fichier modifié à la zone de staging avec git add

`git add` sert à suivre de nouveaux fichiers ajoutés au projet, mais aussi à ajouter dans la zone de staging un fichier déjà suivi par Git.

Par exemple, modifiez le fichier `index.html`, puis exécutez à nouveau :

```
git status
```

Vous verrez alors le fichier apparaître dans la section « **Modifications qui ne seront pas validées** » (*Changes not staged for commit*).

Pour ajouter le fichier à la zone de staging, utilisez la commande :

```
git add index.html
```

Ajouter un fichier modifié à la zone de staging avec git add

```
[tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
[tomezequiel@macbook-air-de-tom Atelier Git % git add index.html
[tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html
```

Maintenant que la **zone de staging** contient les modifications que vous souhaitez enregistrer, vous pouvez créer un commit.

 **Seules les modifications présentes dans la zone de staging seront incluses dans le commit.**

Pour créer le commit, utilisez :

```
git commit
```

Si vous mettez pas l'option -m, la commande ouvre l'éditeur Vim dans la ligne de commandes pour saisir le **message du commit**.

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Committer: Tom Ezequiel <tomezequiel@MacBook-Air-de-Tom.local>
#
# On branch main
# Your branch is up to date with 'origin/main'.
#
# Changes to be committed:
#   modified:   index.html
#
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
```


Git commit -m

On préfère utiliser `git commit` avec l'option `-m`, pour mettre directement le message du commit.

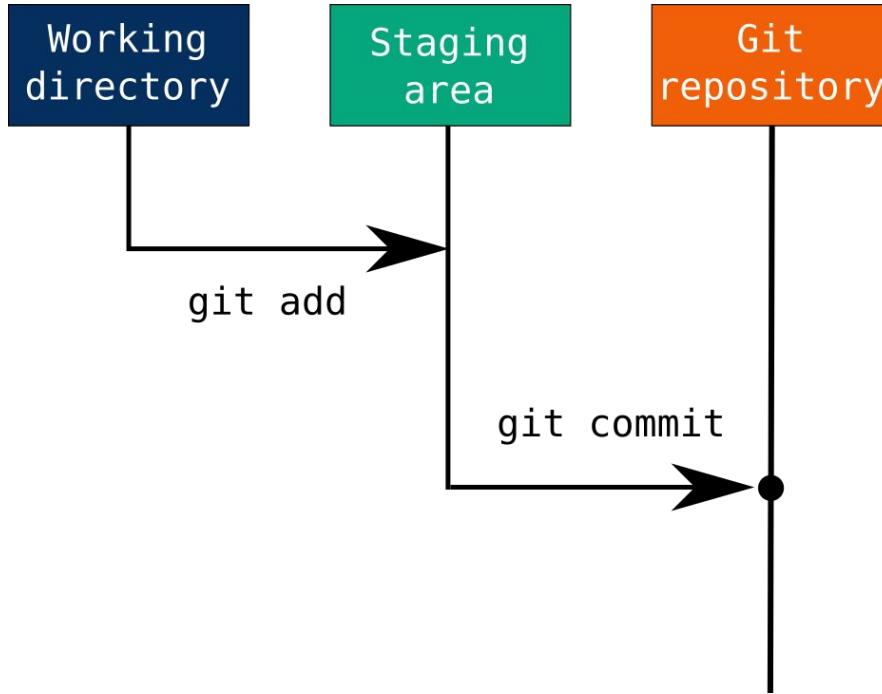
```
[tomezequiel@macbook-air-de-tom Atelier Git % git commit -m "changement titre index.html"]
[main d8fd534] changement titre index.html
1 file changed, 1 insertion(+), 1 deletion(-)
```

Résumé : workflow classique avec Git

Le workflow classique de Git est le suivant :

1. Vous modifiez ou ajoutez des fichiers dans votre **répertoire de travail** : les modifications ne sont pas encore sélectionnées pour le prochain commit.
2. Vous **indexez** les modifications : elles sont ajoutées à la **zone de staging** et seront incluses dans votre prochain commit.
3. Vous **validez** les modifications avec `git commit` : Git enregistre dans le **repository local** l'état correspondant aux modifications présentes dans la zone de staging (ce repository est stocké dans le répertoire caché `.git`).

Résumé : workflow classique avec Git



Git restore

On peut enlever un fichier ajouté à la zone de staging avec la commande :

git restore (nom du fichier)

```
tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")
tomezequiel@macbook-air-de-tom Atelier Git % git restore index.html
tomezequiel@macbook-air-de-tom Atelier Git % git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Lorsque d'autres personnes ont envoyé des modifications sur le dépôt distant, vous pouvez les récupérer dans votre dépôt local avec :

```
git pull
```

 **Pensez à utiliser git pull régulièrement pour que votre dépôt local soit à l'image du dépôt distant !**

Une fois vos modifications **commitées localement**, vous pouvez les envoyer vers le dépôt distant avec :

```
git push
```

Pull avant de push

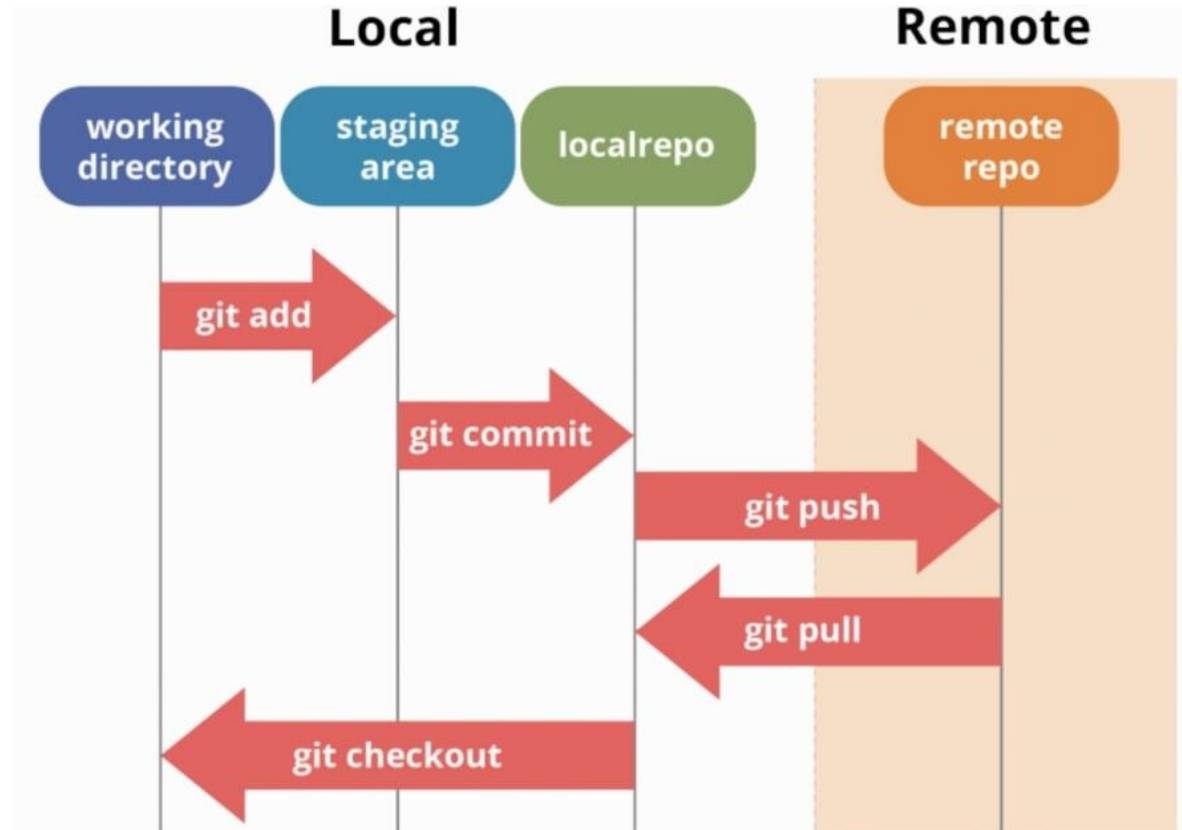
⚠ Si quelqu'un a envoyé des modifications sur cette branche depuis votre dernier `git pull`, le `push` peut être refusé.

Vous devrez d'abord re-récupérer les dernières modifications avec un nouveau `git pull`.

```
[tomezequiel@macbook-air-de-tom Atelier Git % git push
To github.com:tomMizEnseignement/atelierGitNexa15SeptTest.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:tomMizEnseignement/atelierGitNexa15SeptTest.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

Résumé

git checkout était historiquement utilisé pour passer de l'état enregistré vers le working directory. Aujourd'hui, *git restore* c'est *git restore* qui est recommandé.



Branche main

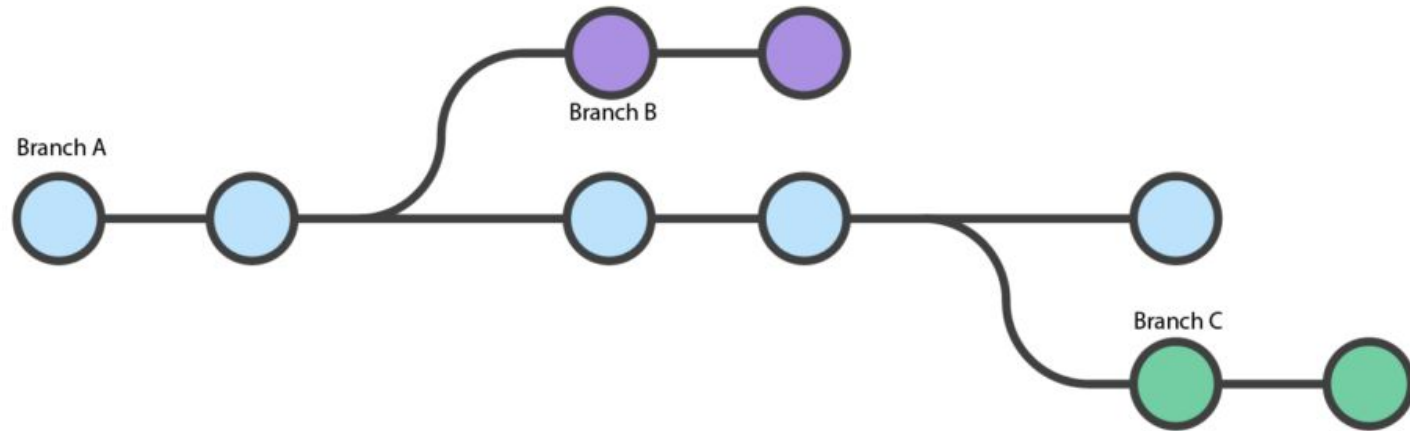
Dans tous les exemples précédents, nous avons travaillé sur la branche principale que Git nous propose qui s'appelle *main*.

Dans un projet collaboratif, on évite généralement de modifier directement *main*.

On crée une **branche** pour travailler sur une fonctionnalité ou une modification particulière.

Une branche est une **ligne de travail indépendante**, qui permet de faire évoluer le projet sans modifier directement *main*.

Les branches



Créer une branche

Pour créer une nouvelle branche et basculer dessus immédiatement :

```
git switch -c nom-de-la-branche
```

Par exemple :

```
git switch -c ajouter-footer
```

Cette commande effectue deux actions :

1. elle **crée** la branche *ajouter-footer* ;
2. elle **bascule** sur cette nouvelle branche.

Pour simplement changer de branche :

```
git switch main
```

À retenir :

```
git switch -c → créer une branche et basculer dessus
```

```
git switch → changer de branche
```

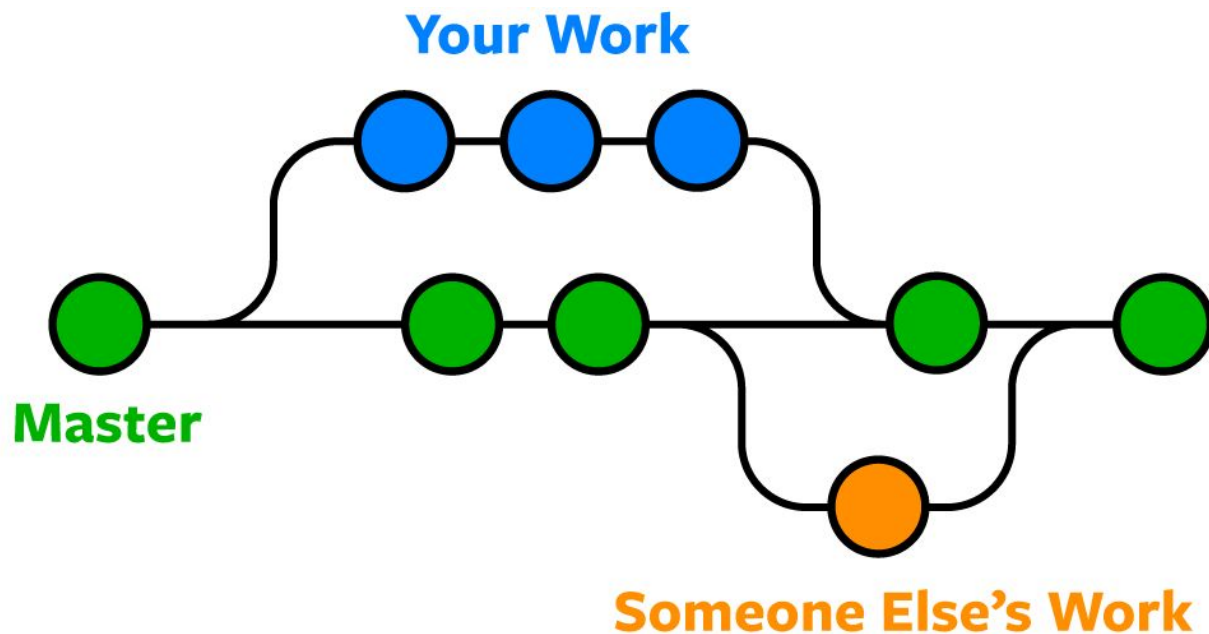
Fusionner une branche

Une fois qu'on a terminé les nouvelles fonctionnalités dans notre branche, il faut la pousser sur le repository distant, avec la commande :

```
git push -u origin nom-de-la-branche
```

Fusionner une branche

Une fois que la branche a été poussée sur le repository distant, on peut la fusionner (merge) avec la branche principale main (parfois appelée Master) grâce à une pull-request



Pull request

1. Créer la Pull Request sur GitHub

Sur GitHub :

1. Ouvrez le repository.
2. Cliquez sur **Compare & pull request** (ou **New pull request** dans l'onglet **Pull requests**).
3. Vérifiez que :
 - **base** = main
 - **compare** = votre branche
4. Ajoutez un titre et une description.
5. Cliquez sur **Create pull request**.

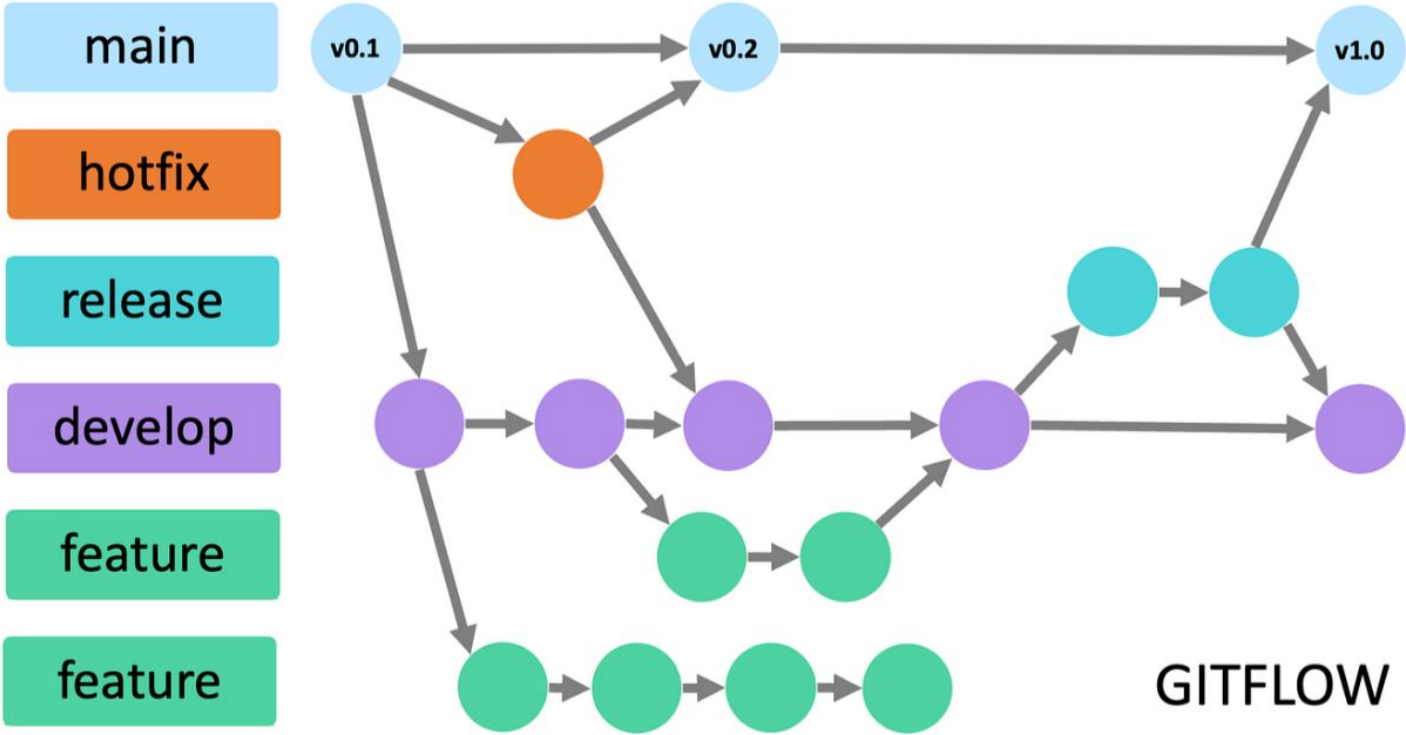
La Pull Request permet aux autres membres de l'équipe de **consulter et vérifier les modifications avant leur fusion dans la branche main**.

2. Accepter et fusionner la Pull Request

Après vérification, un membre autorisé peut cliquer sur **Merge pull request**, puis confirmer avec **Confirm merge**.

GitHub **fusionne alors (merge)** les modifications de la branche dans *main*.

Exemple : un workflow Git en entreprise

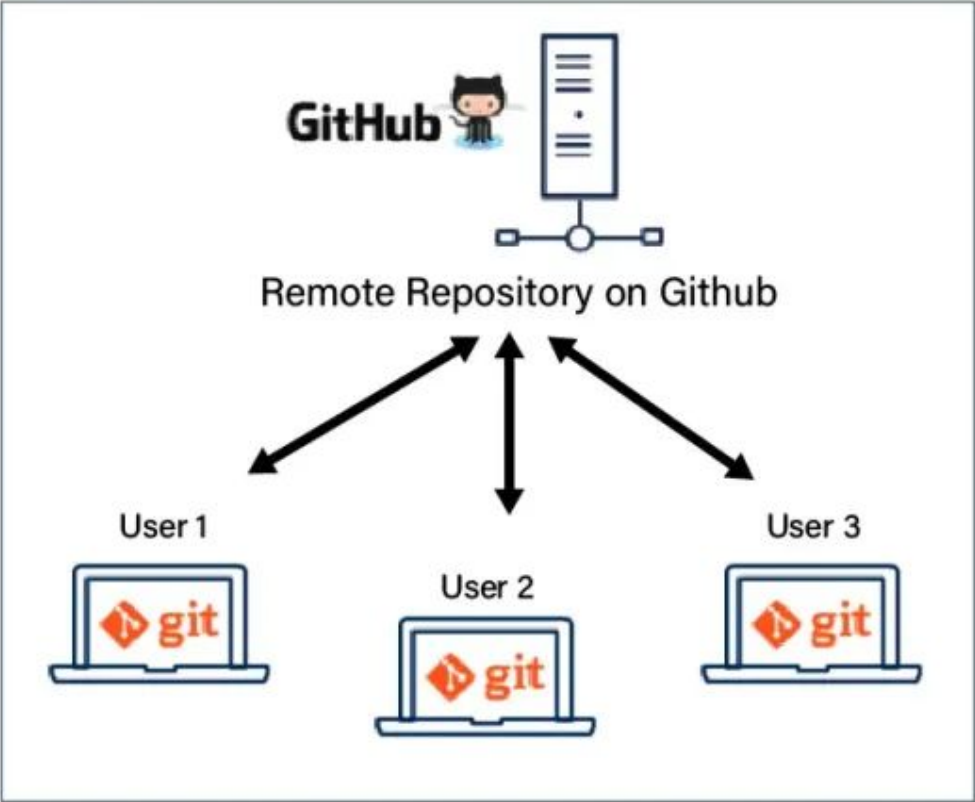


Git permet de gérer l'historique et les versions d'un projet.

GitHub est une plateforme qui permet notamment **d'héberger des dépôts Git à distance** et de collaborer sur ces projets.

Un dépôt GitHub permet notamment de :

- **héberger le code du projet** et son historique ;
- **synchroniser** le travail entre les membres de l'équipe avec *git push* et *git pull* ;
- **travailler à plusieurs** sur différentes branches ;
- créer des **Pull Requests** pour proposer et faire relire des modifications ;



Créer ou utiliser son compte GitHub

Rendez-vous sur GitHub et connectez-vous à votre compte (vous pouvez vous connecter avec votre compte Google).

Si vous n'avez pas encore de compte, créez-en un :

<https://github.com/>

Votre compte sera utilisé pour **identifier vos contributions au projet**.

S'authentifier auprès de GitHub avec SSH

GitHub n'autorise plus l'authentification par **mot de passe** pour les opérations Git.

Pour permettre à votre ordinateur de s'authentifier auprès de GitHub, nous allons utiliser une **clé SSH**.

Le principe repose sur deux clés :

- **clé privée** → reste sur votre ordinateur et ne doit jamais être partagée ;
- **clé publique** → enregistrée sur votre compte GitHub.

Une fois la configuration effectuée, vous pourrez utiliser GitHub avec Git sans avoir à saisir votre mot de passe à chaque opération.

Générer une clé ssh

Premièrement, il faudra générer une clé ssh dans votre terminal (la procédure est indiquée dans le projet) :

```
[tomezequiel@macbook-air-de-tom Atelier Git % cat ~/.ssh/id_ed25519.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHWHeKKhYuC7ZvMaaqkacxDwo3iPVTR9fiUJFvhlyCb4 tomas.mizrahi.enseignement@gmail.com  
tomezequiel@macbook-air-de-tom Atelier Git %
```

Ajouter la clé publique sur Github

Une fois votre paire de clés SSH créées, vous devez **ajouter votre clé publique à votre compte GitHub**.

Dans GitHub :

Settings → **SSH and GPG keys** → **New SSH key**

Collez votre **clé publique** dans le champ prévu à cet effet, puis enregistrez-la.

⚠ Ne partagez jamais votre clé privée.

Seule la clé publique (**.pub**) doit être ajoutée à GitHub.

SSH keys

[New SSH key](#)

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys



SSH

tomas.mizrahi.enseignement@gmail.com

SHA256:bqYJQI4i37/or/TiodrGdMUWV+iVMFJpvimvVbYiJM

Added on Sep 4, 2026

Never used — Read/write

[Delete](#)

Check out our guide to [connecting to GitHub using SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

[New GPG key](#)

Ajouter la clé publique sur github

Add new SSH Key

Title

Key type

Authentication Key ↕

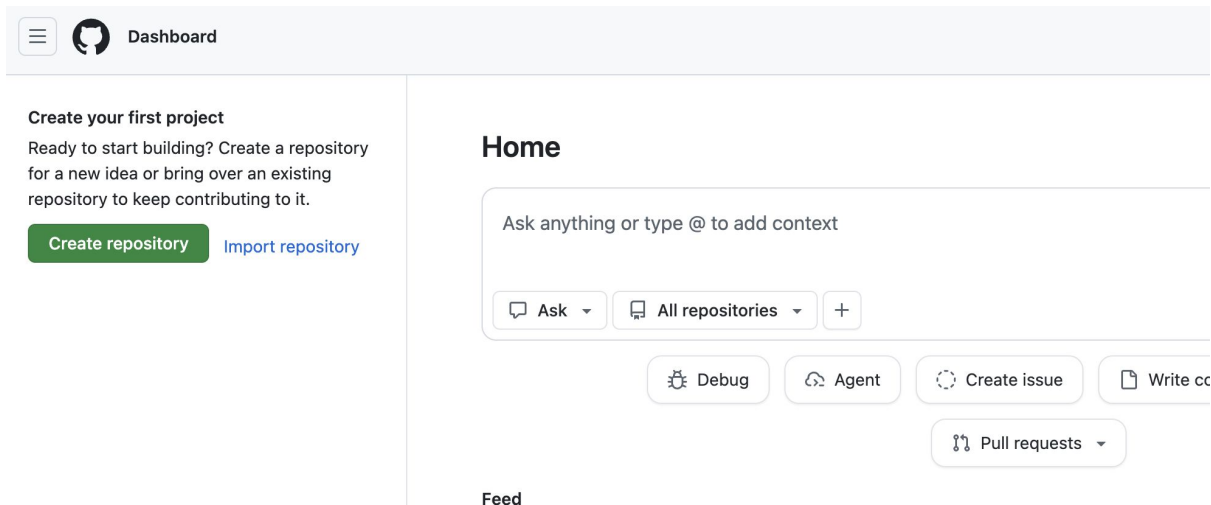
Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

Add SSH key

Création du repository

Pour permettre à Git de gérer notre projet, nous devons créer le **repository distant** de notre projet sur GitHub.



Création du repository

Lors de la création :

- Choisissez un **Repository name**
- Choisissez **Public**
- Ne cochez pas **Add a README file**
- Ne rajoutez pas de `.gitignore` ou de licence pour le moment
- Cliquez sur **Create repository**

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *  tomizDev /

Repository name *

✔ movie-explorer is available.

Great repository names are short and memorable. How about [scaling-palm-tree?](#)

Description

0 / 350 characters

2

Configuration

Choose visibility *  Public

Choose who can see and commit to this repository

Add README Off ☐

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

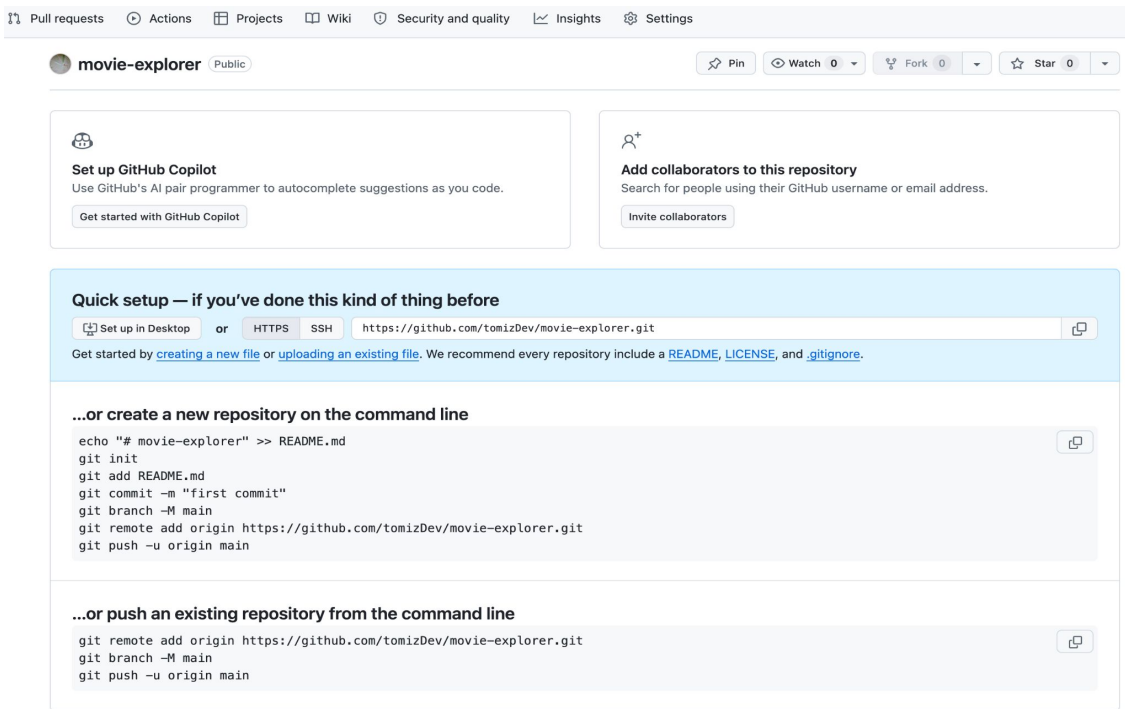
Add license No license

Licenses explain how others can use your code. [About licenses](#)

Create repository

Informations du repository

Une fois que vous créez le repository, vous aurez une page d'informations du repository.



Cloner le repository, lien ssh sur github

Que vous ayez initialisé ou cloné votre dépôt Git local, vous devez le relier à un dépôt distant afin de pouvoir échanger des modifications avec celui-ci :

```
git remote add origin URL_DU_REPOSITORY
```

Par exemple :

```
git remote add origin  
git@github.com:votre-nom/movie-explorer.git
```

origin est le nom donné à votre dépôt distant. C'est le nom utilisé par défaut par Git.

TON ORDINATEUR

dépôt Git local

— origin —→

GITHUB

movie-explorer

The screenshot shows the GitHub repository page for 'movie-explorer'. The 'Local' tab is selected, displaying the 'Clone' button. Below the button, the SSH URL is shown: `git@github.com:tomMizEnseignement/atelierG`. A note indicates to 'Use a password-protected SSH key.' Below the clone options, there are links to 'Open in GitHub Copilot app', 'Open with GitHub Desktop', and 'Download ZIP'. On the right side of the repository page, the repository has 'No description, provided.', '0 stars', '0 watching', and '0 forks'. The 'Releases' section shows 'No releases published' with a link to 'Create a new release'. The 'Packages' section is partially visible at the bottom.